

PROJEKT-DOKUMENTATION



Fachinformatiker für Systemintegration

Dokumentation zur betrieblichen Projektarbeit

Aufbau einer virtuellen IT-Infrastruktur zur Implementierung eines zentralen Monitoring Systems

Abgabetermin: Nürnberg, 15.05.2026

Prüfungsbewerber:
Ioannis Kiourtzidis
Burgschmietstraße 33
90419 Nürnberg

BISSANTZ

Ausbildungsbetrieb:
Bissantz und Company GmbH
Nordring 98
90409 Nürnberg

Inhaltsverzeichnis

Abbildungsverzeichnis	4
Abkürzungsverzeichnis	5
Englisch-Deutsch Übersetzung	6
1. Einleitung	7
1.1 Projektumfeld	7
1.2 Projektziel	7
1.3 Projektbegründung	7
1.4 Projektschnittstellen	7
1.5 Projektabgrenzung	8
2. Projektplanung	8
2.1 Projektphasen	8
2.2 Abweichung vom Projektantrag	9
2.3 Ressourcenplanung	9
2.4 Entwicklungsprozess	9
3. Analysenphase	10
3.1 Ist-Analyse	10
3.2 Wirtschaftlichkeitsanalyse	10
3.2.1 Make-or-Buy-Entscheidung	10
3.2.2 Projektkosten	10
3.2.3 Kostenvergleich	11
3.3 Anwendungsfälle	11
3.4 Qualitätsanforderungen	11
4 Entwurfsphase	12
4.1 Zielplattform	12
4.2 Architekturdesign	12
4.3 Entwurf der überwachten Metriken	13
4.4 Maßnahmen zur Qualitätssicherung	13
5 Implementierungsphase	13
5.1 Aufbau der virtuellen Testumgebung	13
5.2 Installation und Konfiguration Prometheus	14
5.3 Installation der Exporters	15
5.4 Installation und Konfiguration Grafana	15
5.5 Konfiguration des Alertings	15
6 Abnahmephase	17
6.1 Durchgeführte Tests	17
6.2 Abnahme	18

7 Dokumentation.....	18
8 Fazit	18
8.1 Soll-/Ist-Vergleich.....	18
8.2 Nutzwertanalyse: Prometheus/Grafana vs. PRTG.....	19
8.3 Lessons Learned	20
8.4 Ausblick	20
9 Quellen	21
Anleitung.....	22
1. Übersicht.....	22
2. Umgebung	22
3. Netzwerk & VM-Übersicht.....	22
4. Architektur & Ports	22
5. Grafana Installation.....	22
6. Prometheus Installation	23
7. node_exporter (Linux-Systeme).....	24
8. windows_exporter (Windows-Systeme).....	24
9. Blackbox Exporter (Monitoring-Server).....	25
10. Grafana konfigurieren	26
10.1 Prometheus als Datenquelle	26
10.2 Dashboards importieren.....	26
11. Grafana Alerting.....	26
11.1 SMTP-Konfiguration.....	26
11.2 Contact Point	26
11.3 Alert Rules (Beispiele)	26
12. Tests & Überprüfung.....	27
12.1 Service-Status.....	27
12.2 Endpunkte testen	27
13. Fehlerbehebung.....	27
14. Nützliche Befehle	27
15. Erweiterungsmöglichkeiten	27
16. Referenzen	27

Abbildungsverzeichnis

Abbildung 1: Hyper-V-Host Dashboard Visualisierung	15
Abbildung 2: Grafana-Alert bei hoher CPU-Auslastung	16
Abbildung 3: Erfolgreich aufgelöster CPU-Alarm in Grafana	17

Abkürzungsverzeichnis

IIS = Internet Information Services
FTP = File Transfer Protocol
VM = Virtual Machine
HTTP/S = Hypertext Transfer Protocol / Secure
RAM = Random Access Memory
CPU = Central Processing Unit
LTS = Long Term Support
ICMP = Internet Control Message Protocol
BI = Business-Intelligence
JSON = JavaScript Object Notation
YAML = YAML Ain't Markup Language
XML = Extensible Markup Language

Englisch-Deutsch Übersetzung

Scrape = Extrahieren

Network = Netzwerk

Community = Gemeinschaft

Alerting = Alarmierung

Node = Knoten

Exporter = Exporteur

Target = Ziel

Usability = Benutzerfreundlichkeit

Troubleshooting = Fehlerbehebung

1. Einleitung

1.1 Projektumfeld

Die Bissantz & Company GmbH ist ein mittelständisches Softwareunternehmen mit Sitz in Nürnberg sowie weiteren Standorten in Hamburg und Spreitenbach (Schweiz). Das 1996 gegründete Unternehmen beschäftigt rund 100 Mitarbeitende. Der Schwerpunkt liegt auf der Entwicklung und dem Vertrieb von Business-Intelligence-Lösungen. Das Hauptprodukt DeltaMaster ist eine integrierte BI-Software für professionelle Analysten, Berichtsempfänger und Budgetplaner. Sie stellt Ad-hoc-Analysen, betriebswirtschaftliche und statistische Verfahren sowie Data-Mining-Methoden bereit. Zu den Kunden zählen bekannte Unternehmen wie Bosch, Porsche, Leica und Würth.

Das Projekt wurde im Rahmen der Ausbildung zum Fachinformatiker mit der Fachrichtung Systemintegration bei der Bissantz & Company GmbH durchgeführt. Die Initiative dazu ging vom Auszubildenden selbst aus, da er in der bestehenden IT-Infrastruktur des Unternehmens einen konkreten Handlungsbedarf erkannte. Betreut wurde das Projekt vom Ausbilder Onur Dervis.

1.2 Projektziel

Das Projektziel ist die Implementierung einer Monitoring-Lösung mit Prometheus und Grafana in einer virtuellen Testumgebung. Dabei wird ein funktionsfähiges System aufgebaut, das Dashboarding, die Erfassung von Standard- und individuellen Metriken sowie automatisierte E-Mail-Benachrichtigungen umfasst.

Im zweiten Projektschritt wird die Lösung bewertet. Hierzu wird die implementierte Prometheus-/Grafana-Umgebung anhand definierter Kriterien systematisch mit der bestehenden PRTG-Lösung verglichen. Das Ergebnis ist eine dokumentierte Nutzwertanalyse mit Handlungsempfehlung, die der IT-Abteilung als Entscheidungsgrundlage dient.

Das Projekt gilt als erfolgreich, wenn die technische Umsetzung anhand definierter Tests nachweislich funktioniert und eine nachvollziehbare Bewertung der Lösung vorliegt. Das Ziel besteht in der Schaffung einer belastbaren Entscheidungsgrundlage für eine lizenzkostenfreie, flexible und skalierbare Monitoring-Strategie sowie für eine bessere Anpassbarkeit an zukünftige Anforderungen.

1.3 Projektbegründung

Im Unternehmen Bissantz und Company GmbH wird derzeit die kostenlose Lizenzversion von PRTG Network Monitor eingesetzt. Diese ist auf maximal 100 Sensoren begrenzt. Mit zunehmender Systemanzahl wird diese Grenze absehbar erreicht, was eine kostenpflichtige Lizenzerweiterung erforderlich machen würde. Darüber hinaus schränken die begrenzten Anpassungsmöglichkeiten der Dashboards in PRTG die effiziente Nutzung im Arbeitsalltag ein.

Prometheus und Grafana bieten als Open-Source-Alternativen eine unbegrenzte Anzahl an Metriken, ohne dass Lizenzkosten entstehen. Die flexible Dashboard-Gestaltung und die breite Community-Unterstützung sprechen für eine Evaluierung dieser Lösungen als langfristige Monitoring-Strategie.

1.4 Projektschnittstellen

Das Monitoring-System interagiert mit folgenden Systemen und Akteuren:

- Windows 11 VM (Client): Überwachung via Windows Exporter
- Windows Server 2022 VM (FTP-Server): Überwachung via Windows Exporter, FTP-Dienst
- Ubuntu VM (Webserver): Überwachung via Node Exporter, HTTP-Verfügbarkeit via Blackbox Exporter
- Ubuntu-VM (Monitoring-Server): Betrieb von Prometheus und Grafana
- Host-Rechner: physisches System, ebenfalls überwacht
- E-Mail-Dienst: Alerting-Kanal für automatische Benachrichtigungen

Auftraggeber und Benutzer der Lösung ist die IT-Abteilung der Bissantz & Company GmbH. Das Ergebnis wird dem IT-Team präsentiert.

1.5 Projektabgrenzung

Das Projekt umfasst den Aufbau einer Testumgebung sowie die Erstellung einer Dokumentation und einer Anleitung für das IT-Team. Eine Migration des produktiven PRTG-Monitorings in die neue Lösung ist nicht Teil dieses Projekts. Ebenso wird kein Benutzer- oder Rollenmanagement für Grafana konfiguriert. Das Projekt liefert eine Entscheidungsgrundlage, aber keine produktive Umstellung.

2. Projektplanung

2.1 Projektphasen

Das Projekt fand im Zeitraum vom 20.04.2026 bis 24.04.2026 statt. Die tägliche Arbeitszeit betrug 8 Stunden. Die folgende Tabelle zeigt die grobe Zeitplanung:

Projektphase	Geplante Zeit
Projektinitialisierung und Zieldefinition	2,0 Std.
Ist-Analyse PRTG (Metriken, Schwachstellen, Kosten)	2,0 Std.
Soll-Konzept: Monitoring-Strategie, Metrik-Auswahl, Dashboard-Design	3,0 Std.
Recherche Prometheus, Grafana und Exporters	1,0 Std.
Zeit- und Ressourcenplanung	1,0 Std.
Erstellung und Konfiguration von 3 VMs	3,0 Std.
Installation Serverdienste (FTP, Webserver)	2,0 Std.
Installation Linux-VM für Prometheus und Grafana	1,0 Std.
Installation und Konfiguration Prometheus	2,5 Std.
Installation und Konfiguration Grafana	2,0 Std.
Installation Exporters auf allen Systemen	2,0 Std.
Konfiguration Metriken und Scrape-Targets	1,5 Std.
Erstellung individueller Grafana-Dashboards	3,0 Std.
Konfiguration Alert Manager und Alert-Regeln	2,0 Std.
Einrichtung E-Mail-Benachrichtigung	1,0 Std.

Funktions- und Ausfalltests	2,0 Std.
Detaillierte Evaluierung Prometheus/Grafana vs. PRTG	4,0 Std.
Erstellung Projektdokumentation	3,5 Std.
Soll-/Ist-Vergleich	0,5 Std.
Projektübergabe und Präsentation der Empfehlung	1,0 Std.
Gesamt	40,0 Std.

2.2 Abweichung vom Projektantrag

Im ursprünglichen Projektantrag waren folgende Präsentationsmittel vorgesehen, die im Rahmen der tatsächlichen Durchführung angepasst wurden:

Live-Demo: Eine Live-Demo des Monitoring-Systems entfällt aus zeitlichen Gründen. Stattdessen werden die Funktionen und Ergebnisse anhand von Screenshots der laufenden Anwendung demonstriert.

Demonstrationsszenarien: Die simulierten Host-Ausfälle und Alert-Auslösungen werden nicht live vorgeführt, sondern anhand von Screenshots der entsprechenden Grafana-Dashboards und E-Mail-Benachrichtigungen dokumentiert.

ROI-Berechnung: Eine grafische Darstellung der Kostenersparnis bzw. eine ROI-Berechnung entfällt, da es sich bei beiden verglichenen Lösungen — PRTG in der kostenlosen Version sowie Prometheus und Grafana — um kostenfreie Software handelt. Ein monetärer Vergleich ist daher nicht sinnvoll anwendbar.

Grafana-Dashboard-Definitionen: Anstelle der im Antrag genannten JSON-Exporte der Dashboard-Definitionen werden Screenshots der erstellten Grafana-Dashboards beigelegt, die die Überwachung der Systeme und die konfigurierten Panels übersichtlich darstellen.

2.3 Ressourcenplanung

Für die Durchführung des Projekts wurden folgende Ressourcen geplant und eingesetzt:

- Hardware: Host-Rechner mit Intel i7-10700K 3,80GHz CPU, 64GB RAM und 512GB NVMe SSD
- Virtualisierungssoftware: Hyper-V-Manager
- Betriebssysteme: Windows 11, Windows Server 2022, Ubuntu Server 22.04 LTS
- Software: Prometheus, Grafana, Alertmanager, Node Exporter, Windows Exporter, Blackbox Exporter
- Büroarbeitsplatz mit Internetzugang für Recherche und Dokumentation
- E-Mail-Konto für das Alerting-Testing

2.4 Entwicklungsprozess

Das Projekt folgte einem strukturierten Wasserfallmodell mit den Phasen Planung, Analyse, Realisierung und Abschluss. Da es sich um ein klar abgegrenztes Einzelprojekt handelte, war kein iterativer Prozess notwendig. In der Planungsphase erfolgte eine Ist-Analyse und es wurde ein Konzept erstellt. In der Realisierungsphase fand die technische

Umsetzung statt und in der Abschlussphase wurden Tests durchgeführt, eine Evaluation vorgenommen und eine Dokumentation erstellt.

3. Analysenphase

3.1 Ist-Analyse

Die Bissantz & Company GmbH verfügt über eine heterogene IT-Infrastruktur, die aus mehreren virtuellen Maschinen und physischen Systemen besteht und zentral überwacht werden muss. Als Monitoring-Lösung wird aktuell die kostenlose Version von PRTG Network Monitor eingesetzt.

Diese ist auf maximal 100 Sensoren begrenzt. Bei wachsender Infrastruktur wird diese Grenze absehbar erreicht. Ein weiterer Kritikpunkt ist die eingeschränkte Flexibilität der Dashboards: Alle Geräte und Sensoren werden in einer einzigen Gesamtansicht aufgelistet, was die Übersichtlichkeit bei einer steigenden Anzahl von Systemen erheblich reduziert. Die Möglichkeiten zur individuellen Gestaltung der Dashboards sind begrenzt, was das Erkennen kritischer Zustände verzögern kann.

Es besteht daher Bedarf an einer flexibleren, skalierbaren und dauerhaft kostenfreien Monitoring-Lösung.

3.2 Wirtschaftlichkeitsanalyse

3.2.1 Make-or-Buy-Entscheidung

Es wurde geprüft, ob eine kommerzielle Erweiterung von PRTG oder eine andere fertige Lösung alle Anforderungen erfüllt. Die kostenpflichtige PRTG-Lizenz mit unbegrenzter Sensoranzahl verursacht jedoch erhebliche Lizenzkosten. Prometheus und Grafana sind hingegen vollständig Open Source und kostenfrei. Da die funktionalen Anforderungen (unbegrenzte Metriken, flexible Dashboards, E-Mail-Alerting) durch Prometheus und Grafana ohne Lizenzkosten erfüllt werden können, wurde eine Eigenimplementierung gegenüber einer Lizenzerweiterung bevorzugt.

3.2.2 Projektkosten

Die Projektkosten setzen sich aus Personalkosten des Auszubildenden sowie Ressourcenkosten zusammen:

Kostenart	Zeit	Kostensatz	Betrag
Entwicklung / Umsetzung (Auszubildender)	40 Std.	ca. 7,50 €/Std.	300,00 €
Stromkosten (Host-Rechner)	40 Std.	Ca. 0,097 €/Std.	3,88 €
Internetkosten (anteilig)	40 Std.	ca. 1,724 €/Std.	68,96 €
Hardwarekosten	-	Bereits im Betrieb Vorhanden	-
Softwarekosten	-	Free Open Source	-
Gesamt			372,80 €

Formeln:

Betrag: Arbeitszeit × Kostensatz = Betrag

Kostensatz Strom: (Leistung in kW) × (Strompreis €/kWh) = €/Std.

→ 0,248 kW × 0,3918 €/kWh = 0,097 €/Std.

Kostensatz Internet: Monatlicher Bruttobetrag ÷ Monatsstunden = €/Std.

→ z.B. 1258,54 € ÷ 730 Std./Monat = 1,724 €/Std

3.2.3 Kostenvergleich

Beide betrachteten Lösungen sind zum aktuellen Zeitpunkt kostenfrei einsetzbar. PRTG wird in der kostenlosen Version mit einem Limit von 100 Sensoren betrieben, Prometheus und Grafana sind vollständig Open Source ohne jegliche Lizenz einschränkungen.

Sollte die IT-Infrastruktur bei Bissantz & Company GmbH weiterwachsen und das 100-Sensor-Limit von PRTG erreicht werden, würde eine Lizenz Erweiterung auf eine kostenpflichtige Version anfallen. Je nach Lizenzmodell (z.B. PRTG 500 oder höher) belaufen sich diese Kosten laut Hersteller auf mehrere Tausend Euro pro Jahr.

Durch den Einsatz von Prometheus und Grafana entfallen diese zukünftigen Lizenzkosten vollständig, da beide Tools dauerhaft kostenfrei bleiben und keine Sensor-Begrenzung aufweisen. Die einmaligen Projektkosten von ca. 500 € (Personalaufwand und Ressourcen) stehen damit einer langfristigen Kostenvermeidung gegenüber.

3.3 Anwendungsfälle

Das Monitoring-System soll folgende Anwendungsfälle abdecken:

- Systemadministrator überwacht alle VMs und den Host in einer zentralen Oberfläche
- Systemadministrator erhält automatische E-Mail-Benachrichtigung bei Systemausfall oder kritischen Metriken
- Systemadministrator filtert und analysiert historische Metriken in Grafana-Dashboards
- Prometheus extrahiert automatisch Metriken aller Zielsysteme
- Blackbox Exporter prüft periodisch die Erreichbarkeit von Diensten (HTTP, ICMP)

3.4 Qualitätsanforderungen

An das Monitoring-System werden folgende Qualitätsanforderungen gestellt:

Kriterium	Anforderung
Verfügbarkeit	Monitoring-System läuft dauerhaft und ist jederzeit erreichbar
Korrektheit	Metriken müssen akkurat und aktuell sein (Scrape-Intervall: 15 Sek.)
Erreichbarkeit	Grafana-Web Oberfläche ist im Netzwerk erreichbar
Alerting	Bei Ausfall eines überwachten Systems erfolgt E-Mail-Benachrichtigung innerhalb von 1 Minute
Erweiterbarkeit	Neue Systeme können jederzeit als Scrape-Target hinzugefügt werden

Usability	Dashboards sind klar strukturiert und ohne Schulung nutzbar
-----------	---

4 Entwurfsphase

4.1 Zielplattform

Als Zielplattform wurde eine lokale Virtualisierungsumgebung gewählt. Die folgende Übersicht zeigt alle eingesetzten Systeme:

System	Rolle	Betriebssystem	Typ
Monitoring-VM	Prometheus + Grafana + Alertmanager	Ubuntu Server 22.04 LTS	Virtuelle Maschine
Webserver-VM	Apache/Nginx Webserver	Ubuntu Server 22.04 LTS	Virtuelle Maschine
FTP-Server-VM	IIS FTP-Server	Windows Server 2022	Virtuelle Maschine
Client-VM	Windows-Client	Windows 11	Virtuelle Maschine
Host-Rechner	Physisches System	Windows Server 2022	Physisch

4.2 Architekturdesign

Die gewählte Architektur folgt dem Pull-Modell von Prometheus: Der Prometheus-Server fragt in regelmäßigen Abständen (Scrape-Intervall: 15 Sekunden) die konfigurierten Exporters auf den Zielsystemen ab. Die gesammelten Metriken werden lokal gespeichert und über Grafana visualisiert. Der Alertmanager empfängt Alarme von Prometheus und leitet diese per E-Mail weiter.

Komponente	Funktion	Eingesetzt auf
Prometheus	Metriken-Scraping, Speicherung, Alerting-Auslösung	Ubuntu Monitoring-VM
Grafana	Visualisierung, Dashboard-Erstellung	Ubuntu Monitoring-VM
Alertmanager	Alarm-Routing und E-Mail-Versand	Ubuntu Monitoring-VM
Node Exporter	Linux-Systemmetriken (CPU, RAM, Disk, Netzwerk)	Ubuntu VMs
Windows Exporter	Windows-Systemmetriken (CPU, RAM, Disk, Netzwerk)	Windows VMs + Host
Blackbox Exporter	Prüfung der Dienstverfügbarkeit (HTTP, ICMP/Ping)	Ubuntu Monitoring-VM

Die Nutzwertanalyse zur Technologieauswahl ergab folgendes Ergebnis:

Kriterium	Gewichtung	PRTG (kostenlos)	PRTG (kostenpflichtig)	Prometheus/Grafana
Lizenzkosten	5	5 (25)	1 (5)	5 (25)
Skalierbarkeit (Sensoren)	5	2 (10)	5 (25)	5 (25)
Dashboard-Flexibilität	4	3 (12)	3 (12)	4 (12)
Dokumentation/Community	3	3 (9)	3 (12)	4 (12)
Einrichtungsaufwand	3	3 (9)	3 (9)	2 (6)
Alerting-Möglichkeiten	4	3 (12)	5 (20)	5 (20)
Nutzwert (Summe gewichtet)		77	83	100

4.3 Entwurf der überwachten Metriken

Folgende Metriken werden auf allen Systemen erfasst:

- CPU-Auslastung (in %, pro Kern und gesamt)
- Arbeitsspeicher-Auslastung (RAM gesamt, verfügbar, genutzt)
- Festplatten-Auslastung (Speicherplatz, Lese-/Schreibraten)
- Netzwerk-Auslastung (ein- und ausgehender Datenverkehr)
- Erreichbarkeit des Systems (Ping / ICMP via Blackbox Exporter)
- HTTP-Verfügbarkeit des Webserver (via Blackbox Exporter)

4.4 Maßnahmen zur Qualitätssicherung

Zur Sicherung der Qualität wurden folgende Maßnahmen geplant und umgesetzt:

- Funktionstest nach jeder Installationsphase (Exporter erreichbar, Prometheus scraped erfolgreich)
- Ausfalltest: Simulierter VM-Ausfall zur Überprüfung des Alertings
- Überprüfung der E-Mail-Benachrichtigung bei ausgelöstem Alert
- Plausibilitätsprüfung der angezeigten Metriken in Grafana

5 Implementierungsphase

5.1 Aufbau der virtuellen Testumgebung

Zunächst wurden vier virtuelle Maschinen erstellt und mit den benötigten Betriebssystemen installiert: eine Ubuntu-VM als Monitoring-Server, eine Ubuntu-VM als Webserver sowie eine Windows Server 2022-VM als FTP-Server. Zusätzlich wurde eine Windows 11-VM als Client-System eingerichtet. Alle VMs wurden im selben virtuellen Netzwerk betrieben, sodass eine direkte Kommunikation möglich ist.

Auf dem Ubuntu-Webserver wurde ein Apache-Webserver installiert und eine einfache Testwebseite bereitgestellt. Auf dem Windows Server 2022 wurde der IIS FTP-Server eingerichtet und konfiguriert.

5.2 Installation und Konfiguration Prometheus

Prometheus wurde auf der Ubuntu Monitoring-VM als Systemdienst installiert. Die Konfigurationsdatei `prometheus.yml` wurde so angepasst, dass alle Zielsysteme (Scrape-Targets) mit ihren jeweiligen Exporter-Ports und Scrape-Intervallen eingetragen sind. Das globale Scrape-Intervall wurde auf 15 Sekunden gesetzt.

Beispielhafte Struktur der `prometheus.yml`:

```
---
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    basic_auth:
      username: 'admin'
      password: '<PASSWORD>'

    static_configs:
      - targets: ["127.0.0.1:9090"]

  - job_name: "node_exporter"
    static_configs:
      - targets: ["127.0.0.1:9100"]
      labels:
        instance: "Monitoring_srv"
      - targets: ["10.0.0.14:9100"]
      labels:
        instance: "Webclient"

  - job_name: "Windows_VM_Metrics"
    static_configs:
      - targets: ["10.0.0.11:9182"]
      labels:
        instance: "windows_client"
      - targets: ["10.0.0.12:9182"]
      labels:
        instance: "ftp-server"
      - targets: ["10.0.0.10:9182"]
      labels:
        instance: "Hyper-V_Host"

  - job_name: "apache"
    static_configs:
      - targets: ["10.0.0.14:9117"]
      labels:
        instance: "Webclient"

  - job_name: "blackbox"
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
      - targets:
          - http://10.0.0.14
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
```

```
- source_labels: [__param_target]
  target_label: instance
- target_label: __address__
  replacement: 127.0.0.1:9115
```

5.3 Installation der Exporters

Auf den Linux-Systemen (Ubuntu-VMs) wurde der Node Exporter als Systemdienst installiert. Er stellt Systemmetriken standardmäßig auf Port 9100 bereit. Auf den Windows-Systemen (Windows Server 2022, Windows 11 Client, Host-Rechner) wurde der Windows Exporter als Windows-Dienst installiert. Dieser stellt Metriken auf Port 9182 bereit und wird in der Aufgabenplanung automatisch beim Hochfahren des Rechners gestartet. Ebenfalls auf der Monitoring-VM installiert wurde der Blackbox Exporter, der die Überprüfung der Erreichbarkeit von Diensten über HTTP- und ICMP-Probes erlaubt.

5.4 Installation und Konfiguration Grafana

Grafana wurde auf der Monitoring-VM installiert und ist über Port 3000 im Browser erreichbar. Prometheus wurde als Datenquelle eingebunden. Anschließend wurden individuelle Dashboards erstellt, die eine übersichtliche Visualisierung der Metriken je System ermöglichen. Für jedes überwachte System wurde ein eigenes Dashboard-Panel angelegt, das CPU-Auslastung, RAM-Verbrauch, Festplattennutzung und Netzwerk-Traffic anzeigt.

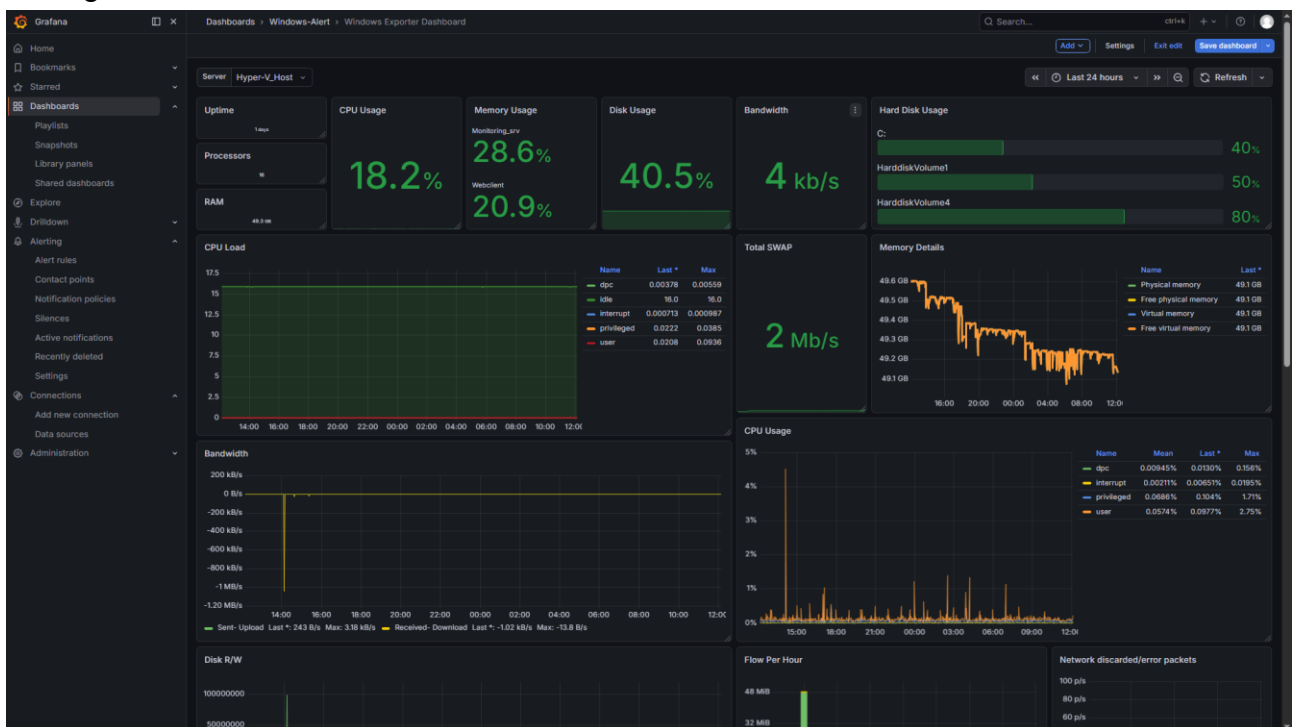


Abbildung 1: Hyper-V-Host Dashboard Visualisierung

5.5 Konfiguration des Alertings

Der Alertmanager wurde als Systemdienst installiert und konfiguriert. In Prometheus wurden Alerting-Regeln definiert, die bei folgenden Ereignissen einen Alarm auslösen:

- Systemausfall: Exporter eines Zielsystems ist länger als 1 Minute nicht erreichbar
- Hohe CPU-Auslastung: CPU-Nutzung liegt über 90 % für mehr als 5 Minuten
- Kritischer Speichermangel: Freier Festplattenspeicher unter 10 %

- Hohe RAM-Auslastung: Arbeitsspeichernutzung über 85 % für mehr als 5 Minuten

Der Alertmanager wurde so konfiguriert, dass bei ausgelöstem Alarm automatisch eine E-Mail an die hinterlegte Adresse versendet wird. Die E-Mail enthält Informationen über das betroffene System, die Art des Alarms sowie den Zeitpunkt der Auslösung.

[FIRING:1] grafana-cpu-metrics Linux - Alerts

ioannis Kiourtzidis

An

ioannis Kiourtzidis

☰

😊

↩️ Antworten

↩️ Allen antworten

➡️ Weiterleiten

👤

⋮

🔔 Wenn Probleme mit der Darstellungsweise dieser Nachricht bestehen, klicken Sie hier, um sie im Webbrowser anzuzeigen.

Linux - Alerts > grafana-cpu-metrics

🔥 1 firing instances

Firing

grafana-cpu-metrics

View alert

Summary

CPU usage has been above 90% for over 5min.

Description

Current CPU usage: 100%

Values

A=99.53333333328676 C=1

Labels

alertname

grafana-cpu-metrics

grafana_folder

Linux - Alerts

Silence

Observed 31s before this notification was delivered, at 2026-05-12 14:24:30 +0200 CEST

Abbildung 2: Grafana-Alert bei hoher CPU-Auslastung

Seite 16 von 27

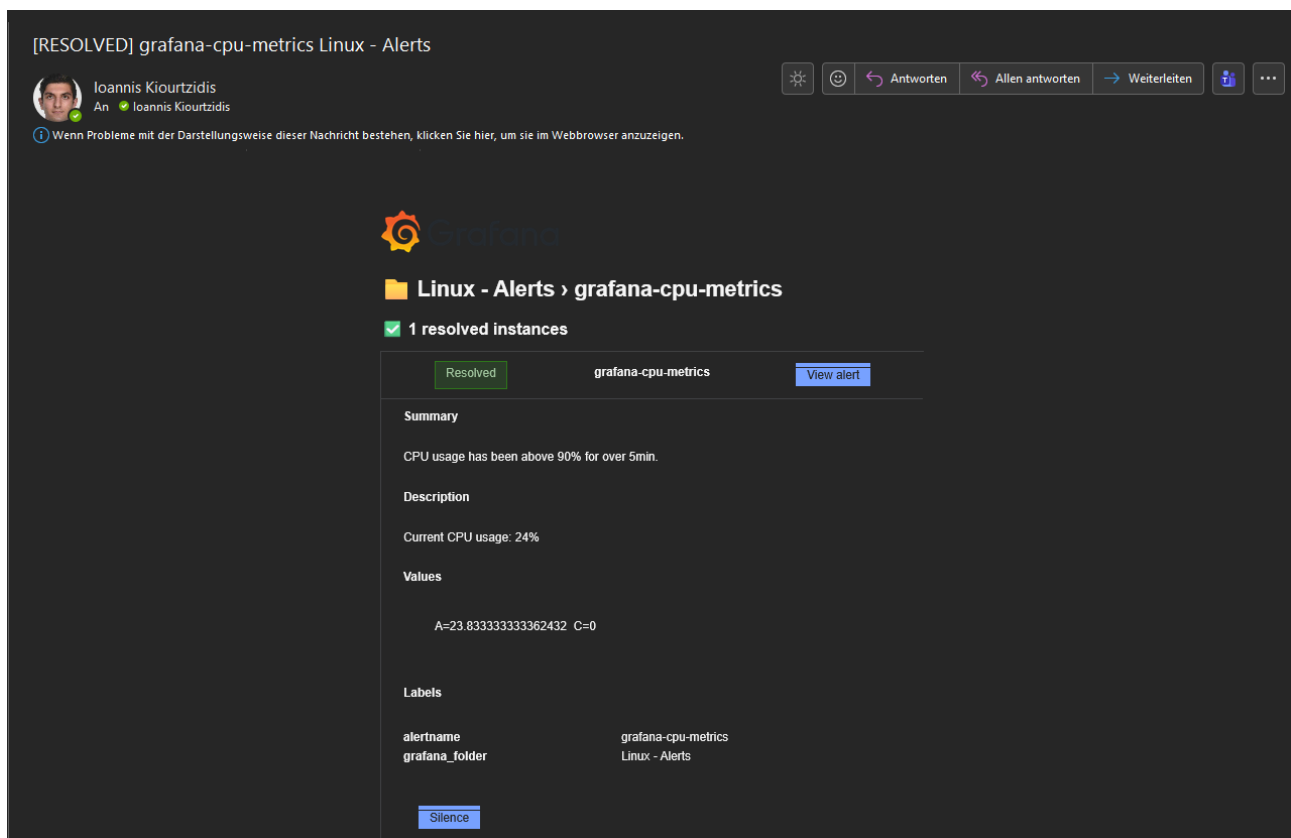


Abbildung 3: Erfolgreich aufgelöster CPU-Alarm in Grafana

6 Abnahmephase

6.1 Durchgeführte Tests

Nach Abschluss der Implementierung wurden folgende Tests durchgeführt:

Testfall	Beschreibung	Ergebnis
Exporter-Erreichbarkeit	Alle Exporter wurden in der Prometheus-Oberfläche auf Status 'UP' geprüft	Bestanden
Metrik-Korrektheit	Angezeigte CPU/RAM-Werte wurden mit Taskmanager / htop verglichen	Bestanden
Dashboard-Anzeige	Alle Grafana-Dashboards wurden auf vollständige und korrekte Datenanzeige geprüft	Bestanden
Ausfallsimulation	Eine VM wurde manuell heruntergefahren und die Alert-Auslösung überprüft	Bestanden
E-Mail-Alerting	Nach VM-Ausfall wurde eine automatische E-Mail-Benachrichtigung empfangen	Bestanden
HTTP-Monitoring	Webserver wurde gestoppt, Blackbox Exporter meldete Nichterreichbarkeit korrekt	Bestanden

Alle definierten Testfälle wurden erfolgreich bestanden. Das Troubleshooting während der Konfiguration von Prometheus und Grafana erforderte zusätzliche Zeit, da es sich um für den Prüfling neue Technologien handelte. Diese Mehrzeit wurde durch die eingeplante Pufferzeit aufgefangen.

6.2 Abnahme

Das aufgebaute Monitoring-System wurde dem IT-Team vorgestellt und demonstriert. Im Rahmen der Präsentation wurden die Dashboards, das E-Mail-Alerting sowie die Nutzwertanalyse präsentiert. Die Abnahme erfolgte auf Basis der nachgewiesenen Funktionsfähigkeit des Systems.

7 Dokumentation

Im Rahmen des Projekts wurde folgende Dokumentation erstellt:

- Projektdokumentation (dieses Dokument): Beschreibung aller Projektphasen für die IHK-Prüfung
- Anleitung für zukünftige Administratoren: Beschreibung der Installation und Konfiguration aller Komponenten, sodass das System reproduziert oder erweitert werden kann
- Konfigurationsdateien: prometheus.yml, Alertmanager-Konfiguration und Alert-Regeln sind kommentiert und als Anlagen beigelegt
- Grafana-Dashboard-Definitionen: Im JSON-Format exportiert für spätere Wiederverwendung

8 Fazit

8.1 Soll-/Ist-Vergleich

Die folgende Tabelle vergleicht die geplante mit der tatsächlich aufgewendeten Zeit:

Phase	Geplant	Tatsächlich	Differenz
Planungsphase	9,0 Std.	9,0 Std.	0 Std.
Realisierungsphase	20,0 Std.	22,0 Std.	+2,0 Std.
Projektabschluss	11,0 Std.	9,0 Std.	-2,0 Std.
Gesamt	40,0 Std.	40,0 Std.	0 Std.

Die Realisierungsphase dauerte aufgrund von Troubleshooting bei der Erstkonfiguration von Prometheus und Grafana etwas länger als geplant. Da es sich um für den Prüfling neue Technologien handelte, war dieser Mehraufwand nicht vollständig vorhersehbar. Die gewonnene Zeit in der Abschlussphase glich dies aus, sodass das Gesamtprojekt planmäßig abgeschlossen werden konnte.

Das Projektziel wurde vollständig erreicht: Das Monitoring-System ist funktionsfähig, überwacht alle definierten Systeme, zeigt die Metriken in benutzerfreundlichen Dashboards an und löst bei kritischen Ereignissen automatische E-Mail-Benachrichtigungen aus.

8.2 Nutzwertanalyse: Prometheus/Grafana vs. PRTG

Die folgende Nutzwertanalyse bewertet beide Lösungen anhand praxisrelevanter Kriterien:

Kriterium	Gewicht	PRTG (kostenlos)	Prometheus/Grafana
Lizenzkosten (langfristig)	5	2 (Limit 100 Sensoren)	5 (vollständig kostenfrei)
Skalierbarkeit	5	1 (100 Sensor-Limit)	5 (unbegrenzt)
Dashboard-Flexibilität	4	2 (begrenzt)	5 (vollständig anpassbar)
Einrichtungsaufwand	3	5 (einfach, GUI)	2 (komplex, Konfigurationsdateien)
Dokumentation/Community	3	4 (gut)	5 (sehr gut, große Community, KI-Unterstützung)
Alerting	4	3 (eingeschränkt)	5 (flexibel konfigurierbar)
Wartungsaufwand	3	4 (gering)	3 (moderat)
Nutzwert (gewichtet)		69 Punkte	108 Punkte

8.3 Lessons Learned

Im Verlauf des Projekts wurden folgende Erkenntnisse gewonnen:

- Prometheus und Grafana erfordern in der Einrichtung deutlich mehr Einarbeitungszeit als grafisch orientierte Tools wie PRTG. Die Konfiguration über YAML-Dateien bietet jedoch eine deutlich höhere Flexibilität.
- Sobald das System korrekt eingerichtet ist, ist die tägliche Nutzung und Erweiterung sehr intuitiv.
- Die umfangreiche offizielle Dokumentation, eine aktive Community sowie der Einsatz von KI-Assistenten (z.B. für Konfigurationsfragen) beschleunigen die Einarbeitung erheblich und reduzieren Fehler.
- Blackbox Exporter ist ein wertvolles Werkzeug zur Überwachung von Dienstverfügbarkeit, das PRTG in der kostenlosen Version nur begrenzt bietet.
- Die Einplanung von Pufferzeit ist bei der Einführung neuer Technologien unbedingt empfehlenswert.

8.4 Ausblick

Auf Basis der Evaluierung wird empfohlen, Prometheus und Grafana als zukünftige Monitoring-Lösung bei Bissantz & Company GmbH einzuführen. Die technische Grundlage ist durch dieses Projekt bereits geschaffen. Als nächste Schritte wären folgende Maßnahmen denkbar:

- Migration der produktiven PRTG-Überwachung auf Prometheus/Grafana
- Erweiterung um weitere Exporters, z.B. für Datenbankdienste oder Anwendungsmonitoring
- Einrichtung von Grafana-Benutzerrollen für unterschiedliche Zugriffsrechte
- Anbindung weiterer Alerting-Kanäle (z.B. Slack, Teams)
- Automatisierte Bereitstellung der Monitoring-Umgebung via Ansible oder Docker Compose

9 Quellen

Grafana Dashboards

- [Template-Dashboards erstellen \(Grafana Docs\)](#)
- [Dashboards importieren \(Grafana Docs\)](#)
- [Grafana & Prometheus auf Ubuntu 24.04 installieren](#)

Alert Manager

- [YouTube-Tutorial zum Alert Manager](#)
- [Grafana Alerting Dokumentation](#)

Netzwerküberwachung

- [YouTube-Tutorial zur Netzwerküberwachung](#)

Dashboard-Syntax (CPU, RAM, Disk)

- [YouTube-Tutorial zu Dashboard-Syntax](#)
- [Prometheus Getting Started Guide](#)

Anleitung

Monitoring-Umgebung: Prometheus, Grafana & Alerting

1. Übersicht

Ziel: Installation von Prometheus, Grafana und Exportern zur Überwachung von Linux- und Windows-Systemen mit integriertem Alerting.

2. Umgebung

Betriebssysteme: Ubuntu Server LTS 22.04/24.04, Windows Server 2022 (21H2), Windows 11 (25H2)

Netzbereich: 10.0.0.0/24

3. Netzwerk & VM-Übersicht

System	IP	Rolle	Komponenten
Monitoring-Server	10.0.0.13	Monitoring	Prometheus, Grafana, Blackbox
Webserver	10.0.0.14	Web + Metrics	Apache, node_exporter
Hyper-V Host	10.0.0.10	Host	windows_exporter
Windows Client	10.0.0.11	Client	windows_exporter
FTP-Server	10.0.0.12	Server	windows_exporter

4. Architektur & Ports

- Prometheus sammelt Metriken von Exporter (Standard: 15s Intervall)
- Grafana visualisiert Daten aus Prometheus
- Grafana Alerting verwaltet Benachrichtigungen

Komponente	Port	Zweck
Prometheus	9090	UI / Scrape
Grafana	3000	Dashboards
node_exporter	9100	Linux Metrics
windows_exporter	9182	Windows Metrics
Blackbox	9115	HTTP Probing

5. Grafana Installation

Abhängigkeiten installieren:

```
apt install gnupg2 apt-transport-https software-properties-common wget
```

Grafana GPG-Key hinzufügen:

```
wget -q -O - https://packages.grafana.com/gpg.key > grafana.key cat grafana.key | gpg --dearmor | tee /etc/apt/trusted.gpg.d/grafana.gpg > /dev/null
```

Grafana Repository hinzufügen:

```
echo 'deb [signed-by=/etc/apt/trusted.gpg.d/grafana.gpg] https://packages.grafana.com/oss/deb stable main' | tee /etc/apt/sources.list.d/grafana.list
```

Grafana Installieren:

```
apt update && apt install -y grafana
```

Service Starten:

```
systemctl daemon-reload systemctl start grafana-server systemctl enable grafana-server
```

Zugriff: `http://10.0.0.13:3000` (Login: admin/admin)

6. Prometheus Installation

Benutzer erstellen:

```
groupadd --system prometheus useradd -s /sbin/nologin --system -g prometheus prometheus
```

Verzeichnisse erstellen:

```
mkdir /var/lib/prometheus for i in rules.d files_sd; do  
mkdir -p /etc/prometheus/${i}; done
```

Neuste Version herunterladen:

```
cd /tmp curl -s https://api.github.com/repos/prometheus/prometheus/releases/latest | grep  
browser_download_url | grep linux-amd64 | cut -d '"' -f 4 | wget -qi -
```

Extrahieren und installieren:

```
tar xzf prometheus*.tar.gz && cd prometheus-* / mv prometheus promtool /usr/local/bin/  
mv prometheus.yml /etc/prometheus/
```

Berechtigungen setzen:

```
for i in rules.d files_sd; do chown -R prometheus:prometheus /etc/prometheus/${i};  
done  
chown -R prometheus:prometheus /var/lib/prometheus/
```

Konfiguration (/etc/prometheus/prometheus.yml):

```
global: scrape_interval: 15s evaluation_interval: 15s scrape_configs: - job_name:  
'prometheus' static_configs: - targets: ['localhost:9090'] - job_name:  
'node_exporter_linux' static_configs: - targets: ['10.0.0.13:9100', '10.0.0.24:9100']  
- job_name: 'windows_exporter' static_configs: - targets: ['10.0.0.10:9182',  
'10.0.0.11:9182', '10.0.0.12:9182'] - job_name: 'blackbox' metrics_path: /probe  
params: module: [http_2xx] static_configs: - targets: ['http://10.0.0.24']  
relabel_configs: - source_labels: [__address__] target_label: __param_target  
- source_labels: [__param_target] target_label: instance - target_label:  
__address__ replacement: 10.0.0.13:9115
```

Systemd Service (/etc/systemd/system/prometheus.service):

```
[Unit]  
Description=Prometheus  
Wants=network-online.target
```

```
After=network-online.target
```

```
[Service]
```

```
Type=simple
```

```
User=prometheus
```

```
Group=prometheus
```

```
ExecStart=/usr/local/bin/prometheus \ --config.file=/etc/prometheus/prometheus.yml \ --
```

```
storage.tsdb.path=/var/lib/prometheus \ --web.listen-address=0.0.0.0:9090
```

```
Restart=always
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Service Starten:

```
systemctl daemon-reload systemctl start prometheus systemctl enable prometheus
```

7. node_exporter (Linux-Systeme)

Auf jedem Linux-System (Monitoring-Server + Webserver):

Herunterladen:

```
cd /tmp curl -s https://api.github.com/repos/prometheus/node_exporter/releases/latest |  
grep browser_download_url | grep linux-amd64 | cut -d '"' -f 4 | wget -qi -
```

Installieren:

```
tar xzf node_exporter*.tar.gz && cd node_exporter-*/ cp node_exporter /usr/local/bin/
```

Service (/etc/systemd/system/node_exporter.service):

```
[Unit]
```

```
Description=Node Exporter
```

```
After=network-online.target
```

```
[Service]
```

```
User=prometheus
```

```
ExecStart=/usr/local/bin/node_exporter
```

```
[Install]
```

```
WantedBy=default.target
```

Service starten:

```
systemctl daemon-reload systemctl start node_exporter systemctl enable node_exporter
```

8. windows_exporter (Windows-Systeme)

Auf jedem Windows-System (PowerShell als Administrator):

- Download: https://github.com/prometheus-community/windows_exporter/releases
- Datei: windows_exporter-0.27.0-amd64.msi

Installation:

```
msiexec /i windows_exporter-0.27.0-amd64.msi
```

```
ENABLED_COLLECTORS="cpu,cs,logical_disk,net,os,system,memory"
```



```
LISTEN_PORT="9182"
```

Firewall-Regel:

```
New-NetFirewallRule -DisplayName "Windows Exporter" -Direction Inbound -Protocol TCP  
-LocalPort 9182 -Action Allow
```

Test: http://<IP>:9182/metrics

9. Blackbox Exporter (Monitoring-Server)

Benutzer erstellen:

```
useradd --no-create-home --shell /bin/false blackbox_exporter
```

Herunterladen und installieren:

```
cd /tmp wget  
https://github.com/prometheus/blackbox_exporter/releases/download/v0.25.0/  
blackbox_exporter-0.25.0.linux-amd64.tar.gz tar xvf blackbox_exporter-*.tar.gz && cd  
blackbox_exporter-*/  
cp blackbox_exporter /usr/local/bin/
```

Verzeichnis erstellen:

```
mkdir /etc/blackbox_exporter  
chown blackbox_exporter:blackbox_exporter /etc/blackbox_exporter
```

Config (/etc/blackbox_exporter/blackbox.yml):

```
modules: http_2xx: prober: http timeout: 5s http: method: GET  
preferred_ip_protocol: "ip4"
```

Service (/etc/systemd/system/blackbox_exporter.service):

```
[Unit]  
Description=Blackbox Exporter  
After=network-online.target  
  
[Service]  
User=blackbox_exporter  
Group=blackbox_exporter ExecStart=/usr/local/bin/blackbox_exporter \ --config.file  
/etc/blackbox_exporter/blackbox.yml  
  
[Install]  
WantedBy=multi-user.target
```

Service starten:

```
systemctl daemon-reload && systemctl start blackbox_exporter && systemctl enable  
blackbox_exporter
```

10. Grafana konfigurieren

10.1 Prometheus als Datenquelle

- Connections → Data Sources → Add data source → Prometheus
- URL: `http://localhost:9090`
- Save & Test

10.2 Dashboards importieren

Dashboard	ID	Verwendung
Node Exporter Full	1860	Linux (CPU, RAM, Disk, Network)
Windows Node Exporter	14510	Windows Monitoring
Blackbox Exporter	13659	HTTP/HTTPS Checks
Prometheus Stats	2	Prometheus Metriken

Import: Dashboards → Import → Dashboard-ID eingeben → Load → Prometheus wählen → Import

11. Grafana Alerting

11.1 SMTP-Konfiguration

Bearbeiten Sie `/etc/grafana/grafana.ini`:

```
[smtp]
enabled = true
host = smtp.beispiel.de:587
user = ihr-username
password = ihr-passwort
from_address = grafana@beispiel.de
from_name = Grafana Monitoring

Grafana neustarten:
systemctl restart grafana-server
```

11.2 Contact Point

Alerts & IRM → Contact points → Add contact point → Integration: Email → Addresses: `ihre-email@beispiel.de`

11.3 Alert Rules (Beispiele)

Alerts & IRM → Alert rules → New alert rule

Instance Down: `up == 0 (For: 1m)`

High CPU: `100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[2m])) * 100) > 80`

High Memory: `(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / node_memory_MemTotal_bytes * 100 > 85`

Low Disk: `(node_filesystem_avail_bytes / node_filesystem_size_bytes) * 100 < 15`

Website Down: `probe_success{job="blackbox"} == 0`

12. Tests & Überprüfung

12.1 Service-Status

```
systemctl status prometheus grafana-server node_exporter blackbox_exporter
```

12.2 Endpunkte testen

- Prometheus: <http://10.0.0.13:9090> → Status → Targets (alle UP)
- Grafana: <http://10.0.0.13:3000>
- Node Exporter: <http://10.0.0.24:9100/metrics>
- Windows: <http://10.0.0.10:9182/metrics>

13. Fehlerbehebung

Target DOWN: Firewall prüfen (ufw allow 9100/tcp), Service-Status, IP/Port in prometheus.yml

Logs: `journalctl -u prometheus -f`

Config testen: `promtool check config /etc/prometheus/prometheus.yml`

Reload: `curl -X POST http://localhost:9090/-/reload`

Grafana keine Daten: Datenquelle-Verbindung testen, Prometheus läuft, Dashboard-Variablen prüfen

14. Nützliche Befehle

Services neu starten:

```
systemctl restart prometheus grafana-server node_exporter
```

Logs anzeigen:

```
journalctl -u prometheus -n 50
```

```
journalctl -u grafana-server -f
```

Metriken testen:

```
curl http://localhost:9100/metrics | head -20
```

```
curl http://10.0.0.10:9182/metrics | grep windows_
```

15. Erweiterungsmöglichkeiten

- Weitere Systeme: Exporter installieren + Target in prometheus.yml hinzufügen
- `apache_exporter` für Webserver-Metriken (Port 9117)
- Weitere Alert-Kanäle: Slack, Microsoft Teams, PagerDuty, Telegram, Discord

16. Referenzen

- Prometheus: <https://prometheus.io/docs/>
- Grafana: <https://grafana.com/docs/>
- Grafana Alerting: <https://grafana.com/docs/grafana/latest/alerting/>
- Grafana Dashboards: <https://grafana.com/grafana/dashboards/>
- Atlantic.net Tutorial: <https://www.atlantic.net/.../how-to-install-grafana-and-prometheus-on-ubuntu-24-04/>